# PoC Plan — Communication architecture for order processing

OrderFlow — Architecture de traitement des commandes · PoC Plan · 2026-10-07 · Statut : seuils fixés avant toute mesure

## Executive conclusion

**Décision candidate : ADOPT EVENT-DRIVEN (provisoire, ADR-001-INITIAL, 2026-10-07).** Ce plan précise comment la mettre à l'épreuve : six scénarios (S1 à S5, plus S4b), des hypothèses dont les seuils sont fixés avant toute mesure, et des règles d'équité pour que seul le style de communication diffère entre les variantes A et B.

**Evidence base : aucune pour l'instant.** Aucun scénario n'a été exécuté. Les verdicts dépendent de trois répétitions concordantes par scénario et par variante.

## 1. Problème

La plateforme doit traiter des commandes dont la charge peut devenir plusieurs fois supérieure à la normale lors des promotions, et dont certaines dépendances (paiement) peuvent ralentir ou devenir indisponibles, **sans jamais confirmer une commande dont le paiement n'est pas autorisé ou le stock pas réservé**. Il faut choisir comment les cinq services (Order, Payment, Inventory, Shipping, Notification) communiquent pour traiter une commande ; aucune donnée ne permet aujourd'hui de dire quel style tient ces contraintes.

**Incertitude.** Nous ne savons pas si le découplage par événements améliore suffisamment la tenue au pic (500 cmd/s) et la tolérance aux pannes de dépendances pour compenser la complexité qu'il ajoute afin de préserver l'invariant (idempotence, outbox, cohérence à terme).

**Assumption centrale.** Nous supposons que l'architecture orientée événements est plus scalable que l'architecture synchrone : sous un pic, la file absorbe la charge et les commandes continuent d'être acceptées dans les délais. La direction technique refuse de l'adopter sans preuve : c'est elle que la PoC expose à la réfutation (H1).

# 2. Options

| Option | Principe | Dans la PoC |
|---|---|---|
| **A — Synchrone** | Order appelle Payment → Inventory → Shipping → Notification en REST et attend chaque réponse | `sync/` |
| **B — Orientée événements** | Order enregistre la commande et publie `OrderCreated` ; chaque service réagit aux événements (chorégraphie, RabbitMQ) | `async/` |
| **Hybride** | Synchrone jusqu'à la confirmation (paiement, stock), événements pour l'expédition et la notification | Hypothèse H6 en brouillon ; preuve à décider (Q-02) |

# 3. Décision candidate

**ADOPT EVENT-DRIVEN** (ADR-001-INITIAL, 2026-10-07, figée avant les mesures). Raisons : la file absorbe les pics de charge, là où la chaîne synchrone sature ; une dépendance lente ou indisponible ne bloque pas la prise en compte des commandes. Elle sera confirmée ou infirmée principalement par H1 (scalabilité) et H3 (résilience), en tenant compte du coût mesuré par H4 (idempotence) et H5 (observabilité).

# 4. Exigences

## 4.1 Exigences de l'énoncé

| Exigence | Valeur | Vérifiée par |
|---|---|---|
| Charge normale | 100 cmd/s sans accumulation significative de travail | S1 |
| Pic promotionnel | 500 cmd/s pendant 5 min, sans effondrement incontrôlé | S2 (H1) |

| Exigence | Valeur | Vérifiée par |
|---|---|---|
| Latence | p95 < 1 s pour la réponse initiale, sous charge normale ; en asynchrone, distinguer latence d'acceptation et latence de complétion | S1 (exigence S1, H2) |
| Fiabilité | Aucune commande confirmée si le paiement ou la réservation du stock a échoué | Tous (I1) |
| Observabilité | Pouvoir reconstruire le traitement d'une commande individuelle | S1 (H5) |

### 4.2 Invariant métier (non négociable)

**I1** — Une commande ne peut être `CONFIRMED` que si le paiement est autorisé **et** le stock réservé. Appliqué en un seul point, par la même requête SQL dans A et B ; compensation : stock insuffisant après paiement autorisé ⇒ annulation du paiement. Dix règles dérivées (I2 à I11 : sûreté et vivacité) vérifient que le système reste cohérent (`docs/invariants.md`).

### 4.3 Précisions de l'équipe

- **Acceptée = paiement passé.** Le critère p95 < 1 s se mesure jusqu'à « acceptée », dans les deux architectures (A-11, A-12).
- **Confirmée = la transaction sur Order**, seul point d'application de l'invariant (A-13).
- **D5.** La confirmation survient dès que le stock est réservé, avant l'expédition. C'est un écart assumé avec la chaîne de l'énoncé, appliqué à l'identique dans A et B ; le workflow de l'énoncé est prévu sur la branche `experiment/workflow-enonce`.

## 5. Hypothèses et mesures de succès

Chaque hypothèse précise la décision évaluée, les conditions, l'attribut de qualité, la mesure et le seuil. Les seuils sont fixés avant toute mesure et ne se modifient pas après avoir vu les résultats.

| # | Attribut | Hypothèse | Scénario | Supportée si | Rejetée si |
|---|---|---|---|---|---|
| **H1** | Scalabilité | Au pic, l'événementiel garde son débit et sa latence d'acceptation | S2 | Sur le palier à 500 cmd/s, B : débit accepté ⩾ 95 % du débit offert **et** acceptation p95 < 1 s **et** 0 violation de I1 ; A manque au moins un de ces seuils | B manque un seuil sans faire mieux que A, ou A tient aussi les trois |
| **H2** | Performance | En charge normale, le synchrone complète plus vite | S1 | Complétion p95 de A au moins 20 % plus faible que celle de B, et acceptation p95 de A < 1 s | Écart < 20 % ou en faveur de B |
| **H3a** | Disponibilité | Un service secondaire en panne n'empêche pas l'événementiel de terminer les commandes | S4b | B : ⩾ 99 % des commandes reçues pendant la panne terminées moins de 2 min après le retour de Shipping | B < 99 %, ou plus de 2 min |
| **H3b** | Disponibilité | Payment en panne : l'événementiel ne perd rien et rattrape | S4 | B : 0 commande perdue et ⩾ 99 % des commandes de la panne dans un état final moins de 2 min après le retour de Payment | Au moins une perdue, < 99 %, ou plus de 2 min |
| **H3c** | Disponibilité | Payment lent : en événementiel, le client n'attend pas | S3 | B : réponse HTTP p95 < 200 ms et 0 abandon ; A : plus de 50 % d'abandons au délai client de 5 s | B ⩾ 200 ms ou avec abandons, ou A ⩽ 50 % |
| **H4** | Cohérence | L'idempotence tient l'invariant à faible coût | S5 | Avec protection : 0 effet en double (I7), 0 violation de I1, surcoût de complétion p95 < 10 % ; le témoin doit montrer des doublons | I7 > 0 ou I1 > 0 avec protection, ou surcoût ⩾ 10 % |

| # | Attribut | Hypothèse | Scénario | Supportée si | Rejetée si |
|---|---|---|---|---|---|
| **H5** | Observabilité | Une commande est plus difficile à reconstruire en événementiel | S1 | B < 99 % de traces complètes et A ⩾ 99 % | B ⩾ 99 % |
| H6 | Performance, disponibilité | L'hybride cumule la rapidité du synchrone et la reprise de l'événementiel | S1, S4b | Seuils en cours de réflexion (hypothèse en brouillon) | — |

**Exigence S1** (verdict séparé, pour A et pour B) : acceptation p95 < 1 s et pas d'accumulation (drainage nul après la charge).

## Règles de verdict

- **Trois répétitions concordantes** par scénario et par variante pour un verdict définitif ; sinon le verdict est **provisoire** ; des répétitions qui se contredisent donnent « non concluante ».
- Une **mesure manquante** donne « non concluante », jamais un succès implicite.
- Une **violation d'invariant** est un résultat à documenter, pas un run à jeter.
- Un refus de paiement ou un stock insuffisant voulus sont des issues métier (`REJECTED`), pas des erreurs techniques.

## Correspondance avec les décisions possibles

| Si les mesures montrent… | Décision |
|---|---|
| H1 et H3 supportées, coût (H4, H5) acceptable | ADOPT EVENT-DRIVEN |
| H1 rejetée (pas d'avantage au pic) mais H3 supportée (résilience) | ADOPT HYBRID ARCHITECTURE (à étayer par H6) |
| Ni H1 ni H3 ne départagent, ou le coût l'emporte | ADOPT SYNCHRONOUS |
| Runs invalides, mesures manquantes, répétitions contradictoires | NO DECISION — additional evidence required |

# 6. Métriques

Une métrique **de verdict** décide ; une métrique **de diagnostic** explique le verdict, elle ne décide rien. Les latences se calculent **depuis la base**, avec les mêmes requêtes pour A et B :

- **acceptation** = `accepted_at − received_at` (paiement passé) ;
- **confirmation** = `confirmed_at − received_at` (stock réservé, invariant satisfait) ;
- **complétion** = `completed_at − received_at` (expédition et notification faites).

| Métrique | Source | Rôle | Hypothèse |
|---|---|---|---|
| Débit offert, itérations perdues | k6 | Validité du run | toutes |
| Débit accepté (commandes acceptées par seconde, par phase) | base | Verdict | H1 |
| Latence d'acceptation p50/p95/p99 | base | Verdict | H1, H2, exigence S1 |
| Latence de complétion p50/p95/p99 | base | Verdict | H2, H4 |
| Latence de réponse HTTP p95, abandons côté client (délai de 5 s) | k6 | Verdict | H3c |
| Commandes perdues, bloquées, terminées ; délai de reprise | k6 + base | Verdict | H3a, H3b |
| Violations I1 à I11 ; commandes en double par requête client | `verify`, base | Verdict | H1, H4, toutes |
| Part de traces complètes | Jaeger | Verdict | H5 |
| Temps de drainage après la charge | base | Diagnostic | H1, H3 |

| Métrique | Source | Rôle | Hypothèse |
|---|---|---|---|
| Profondeur des files (B) ; requêtes en vol, appels en attente de connexion (A) | Prometheus | Diagnostic | H1, H3 |
| Retard de la boucle d'événements, CPU et mémoire par conteneur (y compris PostgreSQL, RabbitMQ, k6) | Prometheus, `docker stats` | Diagnostic (goulot du système ou du banc d'essai ?) | H1, H3c |
| Doublons écartés par la protection | Prometheus | Diagnostic | H4 |

# 7. Scénarios

| ID | Charge | Action | Fenêtre du verdict | Hypothèses |
|---|---|---|---|---|
| **S1** | 100 cmd/s pendant 3 min | aucune | les 3 min | Exigence S1, H2, H5 |
| **S2** | Paliers de 1 min à 100, 200, 300, 400 cmd/s, puis 5 min à 500 cmd/s | aucune | le palier à 500 cmd/s | H1 |
| **S3** | 100 cmd/s pendant 3 min, sonde `GET /orders/:id` une fois par seconde | Payment répond en 2 s pendant 60 s | les 60 s de lenteur | H3c |
| **S4** | 100 cmd/s pendant 3 min | Payment arrêté 30 s, puis redémarré | les 30 s de panne | H3b |
| **S4b** | 100 cmd/s pendant 3 min | Shipping arrêté 30 s, puis redémarré | les 30 s de panne | H3a |

| ID | Charge | Action | Fenêtre du verdict | Hypothèses |
|---|---|---|---|---|
| **S5** | 10 cmd/s pendant 1 min | async : chaque événement livré deux fois ; sync : chaque requête renvoyée avec le même `Idempotency-Key` ; référence sans doublons ; témoin sans protection | la minute mesurée | H4 |

Mélange de charge identique partout : 3 % de paiements refusés, 2 % de commandes sur le produit à stock limité (D12).

## Protocole d'un run ( `./poc scenario …` )

1. Contrôles : un seul projet actif, observabilité vivante, parité sync / async respectée.
2. Réglages du run : traces à 10 % pendant S1 à S4 (100 % en S5), idempotence, doublons.
3. Remise à zéro de la base et des files.
4. **Préchauffage non compté** (1 min), puis charge k6 en modèle ouvert ; la phase est écrite dans chaque commande.
5. Panne posée et levée au bon moment ; écart de plus de 5 s ⇒ run invalide.
6. **Drainage** : attente de la fin de toutes les commandes, plafonnée à 10 min.
7. Invariants, latences par phase, métriques Prometheus, ressources des conteneurs, traces (S1).
8. Validité, puis sortie dans `results/<projet>/<scénario>/rep-<n>/` .

## Validité d'un run

| Statut | Condition | Usage |
|---|---|---|
| Valide | Moins de 1 % d'itérations perdues par k6 | Tout verdict |
| Valide sous saturation | ⩾ 1 % perdues, mais le système sature (VUs tous occupés, abandons ⩾ 1 %) et pas k6 | Seulement pour **rejeter** une hypothèse : moins de charge ne fait qu'avantager le système |

| Statut | Condition | Usage |
|---|---|---|
| Invalide | Sinon | À refaire |

## 8. Règles d'équité

La comparaison n'a de sens que si **seul le style de communication diffère** :

- même logique métier ( `domain.js` identiques, contrôlés par `./poc parity` ) et même invariant, appliqué par la même requête ;
- mêmes ressources : 1 CPU et 512 Mo par service, pool de 20 connexions à la base ; même concurrence par service (64) ;
- même budget de nouvelles tentatives : **0** ; aucune résilience ajoutée à une seule variante ;
- même charge k6, mêmes latences simulées, même délai client (5 s), même échantillonnage des traces ;
- latences mesurées en base avec les mêmes requêtes ; on ne compare jamais l'acceptation d'une variante à la complétion de l'autre.

## 9. Limites connues du dispositif

- Un seul hôte : k6, services, base, broker et observabilité se partagent la machine.
- Une base PostgreSQL unique partagée par les cinq services ; un cœur par service (pas de mise à l'échelle horizontale).
- Services simulés (latences et pannes injectées) ; Payment est un simulateur à réponse immédiate (pas de webhook).
- Orchestration en A, chorégraphie en B : deux choses changent à la fois.
- Budget de nouvelles tentatives à 0 : il pèse sur les résultats de résilience.
- Conclure sur **ce banc d'essai**, pas sur « le synchrone » ou « l'événementiel » en général.

## 10. Livrables associés

| Livrable de l'énoncé | Où |
|---|---|
| PoC Plan | Ce document |

| Livrable de l'énoncé | Où |
|---|---|
| ADR-001 Initial | Document ADR-001-INITIAL (2026-10-07) |
| PoC Implementation | `sync/`, `async/`, `common/`, CLI `./poc` (README) |
| Experimental Evidence | `results/` (non versionné), `docs/resultats.md`, `docs/cas-de-comprehension.md` |
| Architecture Analysis | à rédiger |
| ADR-001 final | `docs/ADR-001.md` |